

RPG Table

1.0

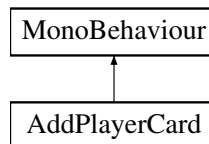
Generated by Doxygen 1.14.0

1 Class Documentation

1.1 AddPlayerCard Class Reference

Manager for player card creation and selection in the game UI.

Inheritance diagram for AddPlayerCard:



Public Member Functions

- void **PlayerCardNotes** ()
Switches to notes view.
- void **PlayerCardCard** ()
Switches to card view.
- void **ShowPlayerCardSelector** ()
Shows the player card selection interface.
- void **Cancel** ()
Cancels card creation and resets UI.
- void **addPlayerCard** ()
Creates a new player card instance.
- void **exitPlayerCardWindow** ()
Closes the player card window.
- void **butClick** (string clickedButton)
Debug method for button click handling.

Public Attributes

- GameObject **PlayerCardWindow**
- GameObject **playerCardSelector**
- GameObject **playerCardDropdown**
- GameObject **playerCardName**
- GameObject **playerButtonPrefab**
- GameObject **playerCardPanel**
- GameObject **playerCardArea**
- GameObject **playerCardCardPanel**
- GameObject **PlayerCardNotesPanel**

1.1.1 Detailed Description

Manager for player card creation and selection in the game UI.

Handles:

- Player card template selection
- Player name assignment
- Card instantiation
- Auto-saving functionality
- Notes/card view switching

1.1.2 Member Function Documentation

addPlayerCard()

```
void AddPlayerCard.addPlayerCard ()
```

Creates a new player card instance.

Validates inputs and:

- Creates player button
- Copies template file
- Sets up click handler

butClick()

```
void AddPlayerCard.butClick (  
    string clickedButton)
```

Debug method for button click handling.

Parameters

<i>clickedButton</i>	Name of clicked button
---	------------------------

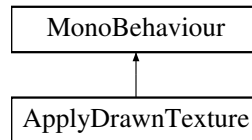
The documentation for this class was generated from the following file:

- C:/Repo/RPG-Table/RPG-Table/Assets/Features/AdditilanScripts/AddPlayerCard.cs

1.2 ApplyDrawnTexture Class Reference

Applies a drawn texture from a **MapBrushDrawer** (p. ??) to a target Renderer's material.

Inheritance diagram for ApplyDrawnTexture:



Public Attributes

- **MapBrushDrawer brushDrawer**
*Reference to the **MapBrushDrawer** (p. ??) component containing the drawn texture.*
- **Renderer planeRenderer**
The Renderer component whose material will receive the drawn texture.

1.2.1 Detailed Description

Applies a drawn texture from a **MapBrushDrawer** (p. ??) to a target Renderer's material.

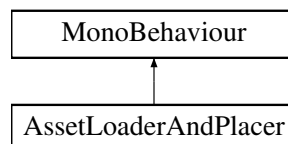
The documentation for this class was generated from the following file:

- C:/Repo/RPG-Table/RPG-Table/Assets/Features/MapDrawingSystem/Scripts/ApplyDrawnTexture.cs

1.3 AssetLoaderAndPlacer Class Reference

Central manager for loading and placing 2D/3D assets in the game environment.

Inheritance diagram for AssetLoaderAndPlacer:



Public Member Functions

- void **restartMaps** ()
Refreshes the map selection panel.
- void **PlaceToken** (string assetName, Vector3 position, Quaternion rotation, Vector3 scale, int layerIndex=0)
Places an asset in the game world.

Public Attributes

- GameObject **imagePrefab**
- GameObject **assetPanel**
- GameObject **asset2DAssetPanel**
- GameObject **asset2DPanel**
- GameObject **asset2Dtext**
- GameObject **texturePanel**
- GameObject **skyboxPanel**
- GameObject **asset3DAssetPanel**
- GameObject **asset3DPanel**
- GameObject **asset3Dtext**
- GameObject **buttonPrefab2D**
- GameObject **buttonPrefab3D**
- GameObject **GameManager**
- GameObject **PlayerCardWindow**
- GameObject **PlayerCardPanel**
- GameObject **PlayerCardNotes**
- Terrain **terrain**
- GameObject **MapPanel**

1.3.1 Detailed Description

Central manager for loading and placing 2D/3D assets in the game environment.

Handles:

- Loading and displaying available assets from directories
- Creating UI buttons for asset selection
- Placing assets in appropriate layers
- Managing 2D/3D mode switching
- Supporting textures, skyboxes, player cards and maps

1.3.2 Member Function Documentation

PlaceToken()

```
void AssetLoaderAndPlacer.PlaceToken (
    string assetName,
    Vector3 position,
    Quaternion rotation,
    Vector3 scale,
    int layerIndex = 0)
```

Places an asset in the game world.

Parameters

<i>assetName</i>	Name of the asset to place
<i>position</i>	World position for placement
<i>rotation</i>	Rotation of the placed asset
<i>scale</i>	Scale of the placed asset
<i>layerIndex</i>	Destination layer (0=Token, 1=Prop, 2=Map)

Handles placement differently based on current game mode (2D/3D) Automatically adds required components (**SmartDragHandler** (p. ??), **AssetName** (p. ??))

restartMaps()

```
void AssetLoaderAndPlacer.restartMaps ()
```

Refreshes the map selection panel.

Clears and repopulates the map panel with current map files

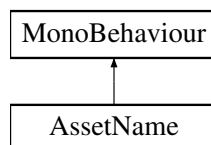
The documentation for this class was generated from the following file:

- C:/Repo/RPG-Table/RPG-Table/Assets/Features/AdditilanScripts/AssetLoaderAndPlacer.cs

1.4 AssetName Class Reference

Simple component for tagging game objects with their asset names.

Inheritance diagram for AssetName:

**Public Attributes**

- string **assetName**
The name of the asset this GameObject represents.

1.4.1 Detailed Description

Simple component for tagging game objects with their asset names.

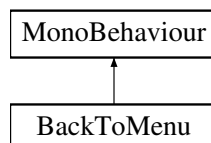
The documentation for this class was generated from the following file:

- C:/Repo/RPG-Table/RPG-Table/Assets/Features/AdditilanScripts/LayerSystem.cs

1.5 BackToMenu Class Reference

Handles navigation back to the main menu scene.

Inheritance diagram for BackToMenu:



Public Member Functions

- void **ReturnToMenu** ()
Loads the "MainMenu" scene when called.

1.5.1 Detailed Description

Handles navigation back to the main menu scene.

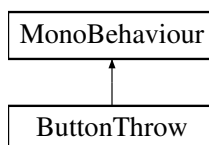
The documentation for this class was generated from the following file:

- C:/Repo/RPG-Table/RPG-Table/Assets/Features/PlayerCardSystem/Scripts/EditorManager.cs

1.6 ButtonThrow Class Reference

A MonoBehaviour class that handles dice throw operations when a button is clicked.

Inheritance diagram for ButtonThrow:



Public Member Functions

- void **makeThrow** ()
Handles the button click event, evaluates the associated dice expression, and logs the result.

Public Attributes

- string **expression** = "2d6"

1.6.1 Detailed Description

A MonoBehaviour class that handles dice throw operations when a button is clicked.

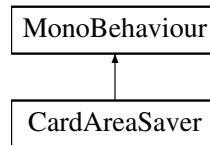
The documentation for this class was generated from the following file:

- C:/Repo/RPG-Table/RPG-Table/Assets/Features/PlayerCardSystem/Scripts/ButtonThrow.cs

1.7 CardAreaSaver Class Reference

System for saving and loading card-based UI layouts with full serialization.

Inheritance diagram for CardAreaSaver:



Public Member Functions

- void **SaveCardArea** ()
Saves the current card area layout to JSON.
- void **LoadCardArea** (bool Editing)
Loads a card area layout from JSON.
- void **ClearCardArea** ()
Clears all objects from the card area.
- void **deleteObject** (string id)
Deletes an object by ID.
- virtual GameObject **CreateChildFromData** (**ChildData** childData, bool dragOn=true)
Creates a UI object from serialized data.
- string **GetSavePath** ()
Gets the current save path.
- void **SetImage** (GameObject obj, string imageName)
Sets an image on a UI object from file.

Static Public Member Functions

- static void **SaveCardArea** (string filePath)
Static method to save card area to specified file path.
- static void **LoadCardArea** (string filePath, bool Editing=true)
Static method to load card area from specified file path.

Public Attributes

- bool **debugLog** = true
Enable debug logging.
- Transform **cardArea**
Parent transform for card area.

Static Public Attributes

- static string **saveFileName**
Header for settings section.
- static string **_fullSavePath**
Complete path for save file.
- static Dictionary< string, GameObject > **_objectDictionary** = new Dictionary<string, GameObject>()
Global dictionary tracking all UI objects by ID.

1.7.1 Detailed Description

System for saving and loading card-based UI layouts with full serialization.

Handles persistence of UI elements including their positions, appearances, and custom behaviors. Supports both editor and runtime usage.

1.7.2 Member Function Documentation

CreateChildFromData()

```
virtual GameObject CardAreaSaver.CreateChildFromData (  
    ChildData childData,  
    bool dragOn = true) [virtual]
```

Creates a UI object from serialized data.

Parameters

<i>childData</i>	Serialized object data
<i>dragOn</i>	Enable drag functionality

Returns

Reconstructed GameObject

deleteObject()

```
void CardAreaSaver.deleteObject (  
    string id)
```

Deletes an object by ID.

Parameters

<i>id</i>	ID of object to delete
-----------	------------------------

GetSavePath()

```
string CardAreaSaver.GetSavePath ()
```

Gets the current save path.

Returns

Current save path

LoadCardArea() [1/2]

```
void CardAreaSaver.LoadCardArea (  
    bool Editing)
```

Loads a card area layout from JSON.

Parameters

☐	<i>Editing</i>	If true, enables editing functionality
--------------------------	----------------	--

Reconstructs the entire UI hierarchy from serialized data. Can operate in either editor or play mode.

LoadCardArea() [2/2]

```
void CardAreaSaver.LoadCardArea (
    string filePath,
    bool Editing = true) [static]
```

Static method to load card area from specified file path.

Parameters

☐	<i>filePath</i>	Path to load file
☐	<i>Editing</i>	Whether to enable editing functionality

SaveCardArea() [1/2]

```
void CardAreaSaver.SaveCardArea ()
```

Saves the current card area layout to JSON.

Captures:

- Object transforms and visibility
- Custom component states
- Button operations
- Visual properties (colors, text, etc.)

SaveCardArea() [2/2]

```
void CardAreaSaver.SaveCardArea (
    string filePath) [static]
```

Static method to save card area to specified file path.

Parameters

☐	<i>filePath</i>	Path to save file
--------------------------	-----------------	-------------------

SetImage()

```
void CardAreaSaver.SetImage (
    GameObject obj,
    string imageName)
```

Sets an image on a UI object from file.

Parameters

	<i>obj</i>	Target object
<i>imageName</i>		Filename of the image

1.7.3 Member Data Documentation

saveFileName

```
string CardAreaSaver.saveFileName [static]
```

Header for settings section.

Filename for saving

The documentation for this class was generated from the following file:

- C:/Repo/RPG-Table/RPG-Table/Assets/Features/PlayerCardSystem/Scripts/SaveLoad.cs

1.8 ChildData Class Reference

Complete data for a child UI element.

Public Member Functions

- **ChildData** (Transform child)
*Constructor that creates **ChildData** (p. ??) from a Transform.*

Public Attributes

- string **objectID**
Unique identifier for the object.
- string **objectType**
Type of UI object.
- string **Text**
Text content.
- string **fontSize**
Font size.
- string **fontColor**
Font color in hex.
- bool **isBold**
Whether text is bold.
- bool **isItalic**
Whether text is italic.
- string **Width**
Width of object.
- string **Height**

- *Height of object.*
- string **backgroundImage**
Background image name.
- string **transparency**
Transparency percentage.
- string **inputType**
Input field type.
- Vector3 **localPosition**
Local position.
- Quaternion **localRotation**
Local rotation.
- Vector3 **localScale**
Local scale.
- bool **isActive**
Whether object is active.
- List< **ScriptData** > **scripts** = new List< **ScriptData**>()
List of attached scripts.
- string **currentOperations**
Current operations (for buttons)

1.8.1 Detailed Description

Complete data for a child UI element.

1.8.2 Constructor & Destructor Documentation

ChildData()

```
ChildData.ChildData (
    Transform child)
```

Constructor that creates **ChildData** (p. ??) from a Transform.

Parameters

<i>child</i>	Transform to serialize
--------------	------------------------

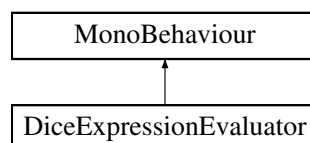
The documentation for this class was generated from the following file:

- C:/Repo/RPG-Table/RPG-Table/Assets/Features/PlayerCardSystem/Scripts/SaveLoad.cs

1.9 DiceExpressionEvaluator Class Reference

A singleton MonoBehaviour that evaluates dice roll expressions in infix notation.

Inheritance diagram for DiceExpressionEvaluator:



Classes

- class **DiceRoll**

Nested class representing a single dice roll with detailed results</summary.

Public Member Functions

- int **EvaluateAndLog** (string infixExpression="0")
Evaluates a dice expression string and returns the result.
- void **throwButton** ()

Public Attributes

- bool **evaluateOnStart** = true
- bool **showPostfixNotation** = true
- bool **detailedDiceLogging** = true
- GameObject **inputField**
- GameObject **MessagePanel**
- GameObject **messagePrefab**

Properties

- static **DiceExpressionEvaluator Instance** [get]

1.9.1 Detailed Description

A singleton MonoBehaviour that evaluates dice roll expressions in infix notation.

Handles complex dice expressions with operators (+, -, *, /, ^, d) and attribute references (@attr). Converts infix notation to postfix, evaluates rolls, and provides detailed logging. Supports features like dice rolling (2d6), math operations, and variable substitution.

1.9.2 Member Function Documentation

EvaluateAndLog()

```
int DiceExpressionEvaluator.EvaluateAndLog (  
    string infixExpression = "0")
```

Evaluates a dice expression string and returns the result.

Parameters

<u>infixExpression</u>	Expression in infix notation (e.g. "2d6+@str")
------------------------	--

Returns

Integer result of the evaluated expression

The documentation for this class was generated from the following file:

- C:/Repo/RPG-Table/RPG-Table/Assets/Features/PlayerCardSystem/Scripts/DiceExpressionEvaluator.cs

1.10 DiceExpressionEvaluator.DiceRoll Class Reference

Nested class representing a single dice roll with detailed results</summary>.

Public Member Functions

- **DiceRoll** (int count, int sides)
- override string **ToString** ()

Public Attributes

- int **DiceCount**
Number of dice rolled.
- int **DiceSides**
Number of sides per die.
- List< int > **Rolls**
List of individual die results.

Properties

- int **Total** [get]
Sum of all dice rolls.

1.10.1 Detailed Description

Nested class representing a single dice roll with detailed results</summary>.

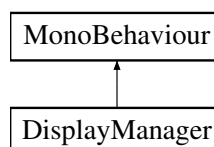
The documentation for this class was generated from the following file:

- C:/Repo/RPG-Table/RPG-Table/Assets/Features/PlayerCardSystem/Scripts/DiceExpressionEvaluator.cs

1.11 DisplayManager Class Reference

Manages multi-display configuration for the application.

Inheritance diagram for DisplayManager:



Public Member Functions

- void **SetPreferredMonitor** (int monitorIndex)
Changes and saves the preferred display setting.

1.11.1 Detailed Description

Manages multi-display configuration for the application.

Handles:

- Remembering preferred display between sessions
- Validating display selection
- Activating chosen display on startup

1.11.2 Member Function Documentation

SetPreferredMonitor()

```
void DisplayManager.SetPreferredMonitor (
    int monitorIndex)
```

Changes and saves the preferred display setting.

Parameters

<code>monitorIndex</code>	Index of the display to use (0-based)
---------------------------	---------------------------------------

- Validates display index exists
- Saves preference to PlayerPrefs
- Note: Requires application restart to take effect

The documentation for this class was generated from the following file:

- C:/Repo/RPG-Table/RPG-Table/Assets/Features/AdditilanScripts/DisplayManager.cs

1.12 DrawingData Class Reference

Serializable data structure for storing drawing information.

Public Attributes

- int **width**
Width of the drawing texture.
- int **height**
Height of the drawing texture.
- string **textureData**
Texture data encoded as Base64 string.
- int **brushSize**
Brush size at time of saving.
- string **brushColor**
Brush color at time of saving as hex string.

1.12.1 Detailed Description

Serializable data structure for storing drawing information.

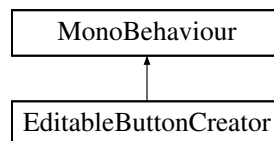
The documentation for this class was generated from the following file:

- C:/Repo/RPG-Table/RPG-Table/Assets/Features/MapDrawingSystem/Scripts/MapBrush.cs

1.13 EditableButtonCreator Class Reference

Creates and manages editable UI buttons with name customization.

Inheritance diagram for EditableButtonCreator:



Public Member Functions

- void **ShowPopupForNewButton** ()
Display popup to create a new button.
- void **SaveButtonName** ()
Saves the current button name from input field.
- void **EditExistingButton** (GameObject buttonToEdit)
Opens popup to edit an existing button's name.

Public Attributes

- GameObject **buttonPrefab**
- GameObject **popupPanel**
- TMP_InputField **nameInputField**
- Transform **buttonsParent**

1.13.1 Detailed Description

Creates and manages editable UI buttons with name customization.

Provides functionality to create, edit, and delete buttons through a popup interface. Buttons can be dynamically added to a specified parent transform.

1.13.2 Member Function Documentation

EditExistingButton()

```
void EditableButtonCreator.EditExistingButton (
    GameObject buttonToEdit)
```

Opens popup to edit an existing button's name.

Parameters

<code>buttonToEdit</code>	The button GameObject to modify
---------------------------	---------------------------------

SaveButtonName()

```
void EditableButtonCreator.SaveButtonName ()
```

Saves the current button name from input field.

Creates new button if none is being edited, otherwise updates the currently edited button's text

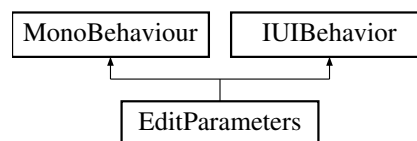
The documentation for this class was generated from the following file:

- C:/Repo/RPG-Table/RPG-Table/Assets/Features/PlayerCardSystem/Scripts/EditorManager.cs

1.14 EditParameters Class Reference

Singleton MonoBehaviour that handles UI element parameter editing.

Inheritance diagram for EditParameters:



Public Member Functions

- void **HandleUIClick** ()
Handles mouse clicks to select UI elements for editing.
- void **textureButtonClick** ()
- void **onEditParametersButtonClick** ()
- void **objUp** ()
Moves the selected object up in the sibling hierarchy.
- void **objDown** ()
Moves the selected object down in the sibling hierarchy.
- void **savePrefab** ()
Saves the current object's parameters as a prefab template.
- void **destroyObject** ()
Destroys the currently selected object.

Public Attributes

- GameObject **objectPanel**
- GameObject **EditParametersPanel**
- GameObject **TextureList**
- GameObject **TexturePanel**
- GameObject **buttonPrefab**

Properties

- static **EditParameters Instance** [get]
Singleton instance accessor.

1.14.1 Detailed Description

Singleton MonoBehaviour that handles UI element parameter editing.

Implements **IUIBehavior** (p. ??) to manage interactive editing of UI elements. Provides comprehensive editing capabilities for position, size, text, appearance, and other parameters of various UI elements.

1.14.2 Member Function Documentation

HandleUIClick()

```
void EditParameters.HandleUIClick ()
```

Handles mouse clicks to select UI elements for editing.

Uses raycasting to detect clicked UI elements and initiates their editing process

Implements **IUIBehavior** (p. ??).

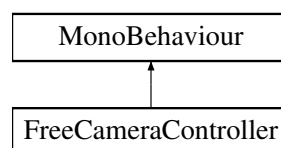
The documentation for this class was generated from the following file:

- C:/Repo/RPG-Table/RPG-Table/Assets/Features/PlayerCardSystem/Scripts/EditParameters.cs

1.15 FreeCameraController Class Reference

Controls free camera movement, rotation, and zoom using mouse and keyboard. Provides first-person style controls with mouse look and keyboard movement.

Inheritance diagram for FreeCameraController:



Public Attributes

- float **moveSpeed** = 50.0f
Base movement speed of the camera in units per second.
- float **rotationSpeed** = 10.0f
Rotation sensitivity for mouse look in degrees per pixel movement.
- float **zoomSpeed** = 50.0f
Zoom speed multiplier for mouse wheel input.
- float **minZoom** = 5f
Minimum allowed zoom distance from the focal point.
- float **maxZoom** = 1000f
Maximum allowed zoom distance from the focal point.
- float **yMinLimit** = -90f
Minimum vertical rotation angle (looking down) in degrees.
- float **yMaxLimit** = 90f
Maximum vertical rotation angle (looking up) in degrees.

1.15.1 Detailed Description

Controls free camera movement, rotation, and zoom using mouse and keyboard. Provides first-person style controls with mouse look and keyboard movement.

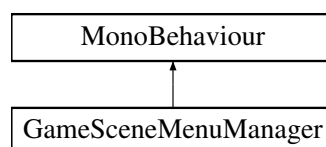
The documentation for this class was generated from the following file:

- C:/Repo/RPG-Table/RPG-Table/Assets/Features/Camera/Scripts/CameraController.cs

1.16 GameSceneManager Class Reference

Manages the UI menu in the game scene, including brush, hammer, dice, and layer settings.

Inheritance diagram for GameSceneManager:



Public Member Functions

- void **ButtonChat** ()
Placeholder for chat functionality.
- void **ButtonExit** ()
Return to the main menu scene.
- void **ButtonPhoto** ()
Placeholder for photo functionality.
- void **ButtonJournal** ()
Placeholder for journal functionality.
- void **ButtonSettings** ()

- Placeholder for settings functionality.*
- void **ButtonBrush** ()
 - Toggle visibility of brush settings panel.*
- void **ButtonRuler** ()
 - Placeholder for ruler functionality.*
- void **TextureButton** ()
 - Show texture selection panel for hammer.*
- void **SkyboxButton** ()
 - Show skybox selection panel.*
- void **ButtonHammer** ()
 - Toggle visibility of hammer settings panel.*
- void **ButtonDice** ()
 - Toggle visibility of dice settings panel.*
- void **ButtonList** ()
 - Toggle visibility of layer selector panel.*
- void **ButtonSend** ()
 - Placeholder for send functionality.*
- void **ButtonLoadArt** ()
 - Placeholder for load art functionality.*
- void **ButtonAddCharacter** ()
 - Placeholder for add character functionality.*
- void **ShowPanel** (GameObject panel)
 - Show the specified panel.*
- void **HidePanel** (GameObject panel)
 - Hide the specified panel.*
- Color **GetBrushColor** ()
 - Get the current brush color.*
- float **GetBrushSize** ()
 - Get the current brush size.*
- void **FindAllButtonsInScene** ()
 - Find all Button objects in the scene and attach sound effects.*

Public Attributes

- GameObject **brushSettingsPanel**
 - Brush Settings.*
- TMP_InputField **hexColorInput**
 - Input field for hex color.*
- Slider **brushSizeSlider**
 - Slider for brush size.*
- GameObject **hammerSettingsPanel**
 - Hammer Settings.*
- Slider **hammerSizeSlider**
 - Slider for hammer size.*
- GameObject **hammerTexturePanel**
 - Panel for hammer texture selection.*
- GameObject **hammerSkyboxPanel**
 - Panel for skybox selection.*
- GameObject **diceSettingsPanel**
 - Dice Settings.*

- GameObject **layerSelectorPanel**

Layer Selector.

- Toggle **toggleGrid**

Toggle for grid layer.

- Toggle **toggleTokens**

Toggle for tokens layer.

- Toggle **toggleFogOfWar**

Toggle for fog of war layer.

- Toggle **toggleEnvironment**

Toggle for environment layer.

- Toggle **toggleBackground**

Toggle for background layer.

1.16.1 Detailed Description

Manages the UI menu in the game scene, including brush, hammer, dice, and layer settings.

1.16.2 Member Function Documentation

ButtonBrush()

```
void GameSceneMenuManager.ButtonBrush ()
```

Toggle visibility of brush settings panel.

Hide other panels

ButtonDice()

```
void GameSceneMenuManager.ButtonDice ()
```

Toggle visibility of dice settings panel.

Hide other panels

ButtonHammer()

```
void GameSceneMenuManager.ButtonHammer ()
```

Toggle visibility of hammer settings panel.

Hide other panels

ButtonList()

```
void GameSceneMenuManager.ButtonList ()
```

Toggle visibility of layer selector panel.

Hide other panels

FindAllButtonsInScene()

```
void GameSceneMenuManager.FindAllButtonsInScene ()
```

Find all Button objects in the scene and attach sound effects.

Find all Button objects in the scene, including inactive ones

Only handle scene-resident buttons, not prefabs

Add special sound for the dice throw button

Default sound for all other buttons

GetBrushColor()

```
Color GameSceneMenuManager.GetBrushColor ()
```

Get the current brush color.

Returns

The brush color or white if invalid

Try to parse hex color input; fallback to white if invalid

GetBrushSize()

```
float GameSceneMenuManager.GetBrushSize ()
```

Get the current brush size.

Returns

The current value of brush size slider

HidePanel()

```
void GameSceneMenuManager.HidePanel (  
    GameObject panel)
```

Hide the specified panel.

Parameters

<i>panel</i>	The panel to hide
--------------	-------------------

ShowPanel()

```
void GameSceneMenuManager.ShowPanel (  
    GameObject panel)
```

Show the specified panel.

Parameters

<code>panel</code>	The panel to show
--------------------	-------------------

1.16.3 Member Data Documentation**brushSettingsPanel**

`GameObject GameManager.SceneMenuManager.brushSettingsPanel`

Brush Settings.

UI panel for brush settings

diceSettingsPanel

`GameObject GameManager.SceneMenuManager.diceSettingsPanel`

Dice Settings.

UI panel for dice settings

hammerSettingsPanel

`GameObject GameManager.SceneMenuManager.hammerSettingsPanel`

Hammer Settings.

UI panel for hammer settings

layerSelectorPanel

`GameObject GameManager.SceneMenuManager.layerSelectorPanel`

Layer Selector.

UI panel for layer visibility toggles

The documentation for this class was generated from the following file:

- `C:/Repo/RPG-Table/RPG-Table/Assets/Features/UI Design/Scripts/GameSceneMenuManager.cs`

1.17 GameSettings Class Reference

Serializable class containing all game settings and paths.

Public Attributes

- string **playerCardsPath** = Path.Combine(Application.persistentDataPath, "PlayerCards/")
Path for player cards storage.
- string **playerCardsPrefabPath** = Path.Combine(Application.persistentDataPath, "PlayerPrefab/")
Path for player card prefabs.
- string **Assets2DPath** = Path.Combine(Application.persistentDataPath, "2DAssets/")
Path for 2D assets.
- string **Assets3DPath** = Path.Combine(Application.persistentDataPath, "3DAssets/")
Path for 3D assets.
- string **GameCardsPath** = Path.Combine(Application.persistentDataPath, "PlayerCardsGame/")
Path for game cards.
- string **MapsPath** = Path.Combine(Application.persistentDataPath, "Maps/")
Path for maps.

1.17.1 Detailed Description

Serializable class containing all game settings and paths.

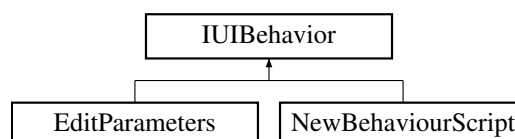
The documentation for this class was generated from the following file:

- C:/Repo/RPG-Table/RPG-Table/Assets/Features/SettingsManager.cs

1.18 IUIBehavior Interface Reference

Interface for UI click handling behavior.

Inheritance diagram for IUIBehavior:



Public Member Functions

- void **HandleUIClick** ()
Method to handle UI click interactions.

1.18.1 Detailed Description

Interface for UI click handling behavior.

1.18.2 Member Function Documentation

HandleUIClick()

```
void IUIBehavior.HandleUIClick ()
```

Method to handle UI click interactions.

Implemented in **EditParameters** (p. ??), and **NewBehaviourScript** (p. ??).

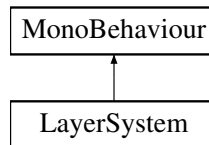
The documentation for this interface was generated from the following file:

- C:/Repo/RPG-Table/RPG-Table/Assets/Features/PlayerCardSystem/Scripts/NewBehaviorScript.cs

1.19 LayerSystem Class Reference

Manages the layer system for 2D and 3D game modes.

Inheritance diagram for LayerSystem:



Public Attributes

- Toggle **tokenLayerToggle**
- Toggle **propLayerToggle**
- Toggle **mapLayerToggle**
- GameObject **token2DLayer**
- GameObject **prop2DLayer**
- GameObject **map2DLayer**
- GameObject **GameManager**
- GameObject **panel**
- GameObject **drawingLayer**
- GameObject **token3DLayer**
- GameObject **prop3DLayer**
- GameObject **map3DLayer**
- GameObject **camera**
- GameObject **table**
- GameObject **drawingButton**
- GameObject **HammerButton**
- string **_GAME_MODE**
- GameObject **_CURRENT_LAYER**

1.19.1 Detailed Description

Manages the layer system for 2D and 3D game modes.

Handles:

- Layer toggling between Token/Prop/Map layers
- 2D/3D mode switching
- Raycast target management for 2D
- Collider management for 3D
- Camera control activation
- UI element visibility

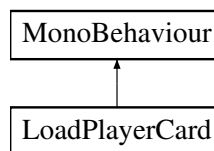
The documentation for this class was generated from the following file:

- C:/Repo/RPG-Table/RPG-Table/Assets/Features/AdditilanScripts/LayerSystem.cs

1.20 LoadPlayerCard Class Reference

Handles loading and saving player card data and associated notes.

Inheritance diagram for LoadPlayerCard:



Public Member Functions

- void **savePlayerCard** (string cardPath, GameObject pca, GameObject pcw, GameObject pcn)
Saves player card data and associated notes.
- void **loadPlayerCard** (string cardPath, GameObject pca, GameObject pcw, GameObject pcn)
Loads player card data and associated notes.
- void **UpdateContentSize** ()
Updates content container size based on children.

Public Attributes

- GameObject **PlayerCardWindow**
- RectTransform **playerCardArea**
- GameObject **playerCardNotes**
- GameObject **manager**

1.20.1 Detailed Description

Handles loading and saving player card data and associated notes.

Manages:

- Player card UI state
- Card content serialization
- Notes storage/retrieval
- Content size calculation

1.20.2 Member Function Documentation

loadPlayerCard()

```
void LoadPlayerCard.loadPlayerCard (
    string cardPath,
    GameObject pca,
    GameObject pcw,
    GameObject pcn)
```

Loads player card data and associated notes.

Parameters

<i>cardPath</i>	Source file path
<i>pca</i>	Player card area reference
<i>pcw</i>	Player card window reference
<i>pcn</i>	Player notes reference

Handles cases where notes file doesn't exist

savePlayerCard()

```
void LoadPlayerCard.savePlayerCard (
    string cardPath,
    GameObject pca,
    GameObject pcw,
    GameObject pcn)
```

Saves player card data and associated notes.

Parameters

<i>cardPath</i>	Destination file path
<i>pca</i>	Player card area reference
<i>pcw</i>	Player card window reference
<i>pcn</i>	Player notes reference

Saves two files:

1. Card content via **CardAreaSaver** (p. ??)
2. Notes as separate JSON file

UpdateContentSize()

```
void LoadPlayerCard.UpdateContentSize ()
```

Updates content container size based on children.

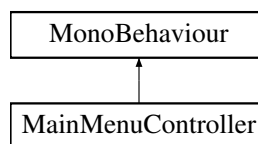
Calculates total height and maximum width of all child elements

The documentation for this class was generated from the following file:

- C:/Repo/RPG-Table/RPG-Table/Assets/Features/AdditilanScripts/LoadPlayerCard.cs

1.21 MainMenuController Class Reference

Inheritance diagram for MainMenuController:



Public Member Functions

- void **StartGame** ()
- void **JoinGame** ()
- void **CardEditor** ()
- void **OpenOptions** ()
- void **ExitGame** ()

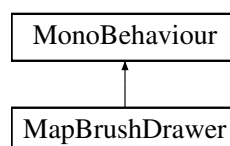
The documentation for this class was generated from the following file:

- C:/Repo/RPG-Table/RPG-Table/Assets/Features/UI Design/Scripts/MainMenuController.cs

1.22 MapBrushDrawer Class Reference

Enables drawing, erasing, and saving/loading textures on a UI canvas.

Inheritance diagram for MapBrushDrawer:



Public Member Functions

- void **ToggleDrawing** ()
Toggles drawing mode on/off. Automatically disables erasing mode when activated.
- void **ToggleErasing** ()
Toggles erasing mode on/off. Automatically disables drawing mode when activated.
- void **ClearTexture** ()
Clears the entire drawing surface to transparent.
- Texture2D **GetDrawnTexture** ()
Gets the current drawing texture.
- void **SaveDrawingToFile** (string filePath)
Saves the current drawing to a file in JSON format.
- void **LoadDrawingFromFile** (string filePath)
Loads a drawing from a file and applies it to the drawing surface.

Public Attributes

- RawImage **drawingSurface**
The RawImage component that serves as the drawing surface.
- Texture2D **brushTexture**
Texture used as the brush stamp (unused in current implementation).
- int **brushSize** = 16
Current size of the brush in pixels.
- Color **brushColor** = Color.black
Current color of the brush.
- GameObject **brushSizeColorField**
Input field for setting brush color via hex code.
- Slider **brushSizeSlider**
Slider control for adjusting brush size.
- bool **isDrawing** = false
Flag indicating whether drawing mode is active.
- Toggle **ErasingToggle**
Toggle control for erasing mode.
- bool **isErasing** = false
Flag indicating whether erasing mode is active.
- GameObject **scrollView**
ScrollView container for the drawing surface.

1.22.1 Detailed Description

Enables drawing, erasing, and saving/loading textures on a UI canvas.

1.22.2 Member Function Documentation

GetDrawnTexture()

```
Texture2D MapBrushDrawer.GetDrawnTexture ()
```

Gets the current drawing texture.

Returns

The Texture2D containing the current drawing.

LoadDrawingFromFile()

```
void MapBrushDrawer.LoadDrawingFromFile (
    string filePath)
```

Loads a drawing from a file and applies it to the drawing surface.

Parameters

<code>filePath</code>	Full path to the source file.
-----------------------	-------------------------------

SaveDrawingToFile()

```
void MapBrushDrawer.SaveDrawingToFile (
    string filePath)
```

Saves the current drawing to a file in JSON format.

Parameters

<code>filePath</code>	Full path to the destination file.
-----------------------	------------------------------------

The documentation for this class was generated from the following file:

- C:/Repo/RPG-Table/RPG-Table/Assets/Features/MapDrawingSystem/Scripts/MapBrush.cs

1.23 MapInfo Class Reference

Container for map metadata and content.

Public Attributes

- string **saveTime**
- string **mapType**
- string **terrainTexturePath**
- bool **terrainFill**
- string **skyboxTexturepath**
- List< **ObjectData** > **tokenLayer**
- List< **ObjectData** > **propLayer**
- List< **ObjectData** > **mapLayer**

1.23.1 Detailed Description

Container for map metadata and content.

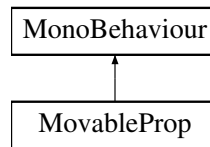
The documentation for this class was generated from the following file:

- C:/Repo/RPG-Table/RPG-Table/Assets/Features/AdditilanScripts/SaveLoadMap.cs

1.24 MovableProp Class Reference

A component for handling a movable and interactive prop in the scene, supporting dragging, rotating, scaling, and bloom/emission lighting control.

Inheritance diagram for MovableProp:



Public Member Functions

- void **OnDrag** (Vector3 newPos)
Moves the object to a new position.
- void **OnRotate** (Vector3 axis, float angle)
Rotates the object around a given axis.
- void **OnScale** (Vector3 newScale)
Sets the local scale of the object.
- Vector3 **GetPosition** ()
Gets the current world position of the object.
- Vector3 **GetScale** ()
Gets the current local scale of the object.
- float **GetIntensity** ()
Gets the current emission/light intensity.
- void **ToggleBloom** ()
Toggles the bloom/emission effect on or off. Updates material emission and light state accordingly.
- void **SetEmissionColor** (Color color)
Sets the emission color for both the material and the point light.
- void **SetIntensity** (float power)
Adjusts the emission/light intensity by a delta value.

1.24.1 Detailed Description

A component for handling a movable and interactive prop in the scene, supporting dragging, rotating, scaling, and bloom/emission lighting control.

1.24.2 Member Function Documentation

GetIntensity()

```
float MovableProp.GetIntensity ()
```

Gets the current emission/light intensity.

Returns

Current intensity value.

GetPosition()

```
Vector3 MovableProp.GetPosition ()
```

Gets the current world position of the object.

Returns

World position of the transform.

GetScale()

```
Vector3 MovableProp.GetScale ()
```

Gets the current local scale of the object.

Returns

Local scale of the transform.

OnDrag()

```
void MovableProp.OnDrag (  
    Vector3 newPos)
```

Moves the object to a new position.

Parameters

<code>newPos</code>	The target world position.
---------------------	----------------------------

OnRotate()

```
void MovableProp.OnRotate (  
    Vector3 axis,  
    float angle)
```

Rotates the object around a given axis.

Parameters

<code>axis</code>	The axis to rotate around.
<code>angle</code>	The angle in degrees to rotate.

OnScale()

```
void MovableProp.OnScale (  
    Vector3 newScale)
```

Sets the local scale of the object.

Parameters

<code>newScale</code>	New local scale vector.
-----------------------	-------------------------

SetEmissionColor()

```
void MovableProp.SetEmissionColor (  
    Color color)
```

Sets the emission color for both the material and the point light.

Parameters

<code>color</code>	New emission color.
--------------------	---------------------

SetIntensity()

```
void MovableProp.SetIntensity (  
    float power)
```

Adjusts the emission/light intensity by a delta value.

Parameters

<code>power</code>	Amount to add to current intensity (clamped to non-negative).
--------------------	---

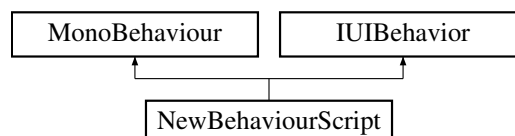
The documentation for this class was generated from the following file:

- C:/Repo/RPG-Table/RPG-Table/Assets/Features/BoardEditor/MovableProp.cs

1.25 NewBehaviourScript Class Reference

Singleton manager for handling button operations and expressions in the UI.

Inheritance diagram for NewBehaviourScript:



Public Member Functions

- string **GetOperationsForObject** (string objectId)
Retrieves operations string for a specific object ID.
- void **HandleUIClick** ()
Handles UI click events to select buttons for editing.
- void **cancelEditMode** ()
- void **loadOperations** (string id, string operations)
- void **LoadOperationsForCurrentButton** ()
Loads operations for the currently selected button.
- void **plusButton** ()
Adds a mathematical operator to the current operations.
- void **minusButton** ()
- void **timesButton** ()
- void **divideButton** ()
- void **addButton** ()
Shows the value addition selector UI.
- void **addValueButton** ()
Adds a numeric or ID reference value to the operations.
- void **addValueCancel** ()
- void **confirmButton** ()
Saves the current operations to the selected button.
- void **clearDictionary** ()
Clears all stored operations from the dictionary.
- void **deleteKey** (string id)
Removes operations for a specific ID from the dictionary.

Properties

- static **NewBehaviourScript Instance** [get]
Singleton instance accessor.

1.25.1 Detailed Description

Singleton manager for handling button operations and expressions in the UI.

Implements **IUIBehavior** (p.??) to manage interactive editing of button expressions. Maintains a dictionary of button operations and provides UI for editing them. Supports mathematical operations and references to other UI elements.

1.25.2 Member Function Documentation

deleteKey()

```
void NewBehaviourScript.deleteKey (  
    string id)
```

Removes operations for a specific ID from the dictionary.

Parameters

<code>id</code>	The ID to remove
-----------------	------------------

GetOperationsForObject()

```
string NewBehaviourScript.GetOperationsForObject (
    string objectId)
```

Retrieves operations string for a specific object ID.

Parameters

<code>objectId</code>	The ID of the object to look up
-----------------------	---------------------------------

Returns

Operations string if found, null otherwise

HandleUIClick()

```
void NewBehaviourScript.HandleUIClick ()
```

Handles UI click events to select buttons for editing.

Activates when in edit mode and detects clicks on button objects. Updates the UI to show the selected button's current operations.

Implements **IUIBehavior** (p. ??).

The documentation for this class was generated from the following file:

- C:/Repo/RPG-Table/RPG-Table/Assets/Features/PlayerCardSystem/Scripts/NewBehaviorScript.cs

1.26 ObjectData Class Reference

Serialized data for individual map objects.

Public Attributes

- string **assetName**
- Vector3 **position**
- Quaternion **rotation**
- Vector3 **scale**
- float[,] **heightMap**
- bool[,] **holeMap**

1.26.1 Detailed Description

Serialized data for individual map objects.

The documentation for this class was generated from the following file:

- C:/Repo/RPG-Table/RPG-Table/Assets/Features/AdditilanScripts/SaveLoadMap.cs

1.27 ObjectDictData Class Reference

Complete serialization data for a UI element.

Public Attributes

- string **objectID**
Object identifier.
- string **objectName**
Object name.

1.27.1 Detailed Description

Complete serialization data for a UI element.

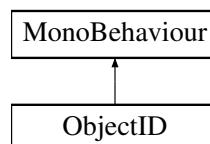
The documentation for this class was generated from the following file:

- C:/Repo/RPG-Table/RPG-Table/Assets/Features/PlayerCardSystem/Scripts/SaveLoad.cs

1.28 ObjectID Class Reference

Unique identifier component for GameObjects with dictionary management.

Inheritance diagram for ObjectID:



Public Member Functions

- void **SetID** (string newId, GameObject objectRef, string prefabName=null)
Sets the object's ID and registers it in the global dictionary.
- string **GetID** ()
Gets the object's unique ID.
- string **GetPrefab** ()
Gets the object's prefab name.

Static Public Member Functions

- static bool **IDExists** (string idToCheck)
Checks if an ID exists in the global dictionary.
- static void **printDictionary** ()
Debug utility to print all objects in dictionary.
- static GameObject **GetObjectByID** (string id)
Retrieves a GameObject by its ID.
- static Dictionary< string, GameObject > **GetAllObjects** ()
Gets all objects in the global dictionary.
- static void **RemoveObjectByID** (string id)
Removes an object from the dictionary by ID.
- static void **Clear_objectDictionary** ()

Public Attributes

- string **_id**
The unique identifier for this object.
- string **_prefabName**
The prefab name this object was created from.

1.28.1 Detailed Description

Unique identifier component for GameObjects with dictionary management.

Tracks objects in a central dictionary using unique IDs. Provides static methods for object lookup and management. Integrates with **CardAreaSaver** (p. ??) for dictionary storage.

1.28.2 Member Function Documentation

GetAllObjects()

```
Dictionary< string, GameObject > ObjectID.GetAllObjects () [static]
```

Gets all objects in the global dictionary.

Returns

Dictionary of all registered objects

GetObjectByID()

```
GameObject ObjectID.GetObjectByID (  
    string id) [static]
```

Retrieves a GameObject by its ID.

Parameters

<code>id</code>	ID to look up
-----------------	---------------

Returns

GameObject if found, null otherwise

IDExists()

```
bool ObjectID.IDExists (
    string idToCheck) [static]
```

Checks if an ID exists in the global dictionary.

Parameters

<code>idToCheck</code>	ID to verify
------------------------	--------------

RemoveObjectByID()

```
void ObjectID.RemoveObjectByID (
    string id) [static]
```

Removes an object from the dictionary by ID.

Parameters

<code>id</code>	ID of object to remove
-----------------	------------------------

Also destroys the GameObject instance

SetID()

```
void ObjectID.SetID (
    string newId,
    GameObject objectRef,
    string prefabName = null)
```

Sets the object's ID and registers it in the global dictionary.

Parameters

<code>newId</code>	Unique identifier to assign
<code>objectRef</code>	Reference to the GameObject
<code>prefabName</code>	Optional prefab name (uses object name if null)

Will destroy the object if ID already exists in the dictionary

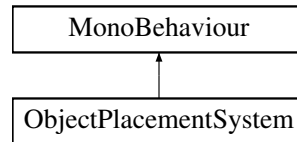
The documentation for this class was generated from the following file:

- C:/Repo/RPG-Table/RPG-Table/Assets/Features/PlayerCardSystem/Scripts/ObjectID.cs

1.29 ObjectPlacementSystem Class Reference

System for placing and managing UI objects in a card-based interface.

Inheritance diagram for ObjectPlacementSystem:



Classes

- class **PlaceableObject**
Serializable class representing a placeable object.

Public Member Functions

- void **EnterDeleteMode** ()
Enters object deletion mode.
- void **SelectObject** (GameObject prefab, **savingCustomObjects** obj=null)
Selects an object for placement and creates a preview instance.
- void **ConfirmPlacement** ()
Finalizes placement of the current object with assigned ID.
- void **ConfirmDeletion** ()
Confirms deletion of selected object.
- void **CancelDeletion** ()
Cancels current deletion operation.
- void **printDictionary** ()
- void **CancelPlacement** ()
Cancels current placement operation.
- void **onCustomPrefabCreatorButtonClick** ()
- void **onCustomPrefabCreatorCloseButtonClick** ()
- void **onCustomPrefabSaveButtonClick** (**savingCustomObjects** customObject)
Saves a custom object configuration.

Static Public Member Functions

- static void **SetObjectComponentsEnabled** (GameObject obj, bool enabled)
*Enables/disables all components on an object except **ObjectID** (p. ??).*

Public Attributes

- List< **PlaceableObject** > **availableObjects**
- List< **savingCustomObjects** > **userObjects**
- Transform **objectsPanel**
- Transform **cardArea**
- GameObject **objectButtonPrefab**
- GameObject **confirmationDialog**
- GameObject **CustomPrefabCreator**
- GameObject **inputTypeDropdown**
- InputField **idInputField**
- GameObject **deleteConfirmationDialog**

1.29.1 Detailed Description

System for placing and managing UI objects in a card-based interface.

Handles object selection, placement, configuration and persistence. Manages both predefined and user-created prefabs with full customization. Integrates wit

1.29.2 Member Function Documentation

ConfirmDeletion()

```
void ObjectPlacementSystem.ConfirmDeletion ()
```

Confirms deletion of selected object.

Updates save file and refreshes UI

ConfirmPlacement()

```
void ObjectPlacementSystem.ConfirmPlacement ()
```

Finalizes placement of the current object with assigned ID.

Handles special configuration for input fields Adds **ObjectID** (p. ??) component if missing

onCustomPrefabSaveButtonClick()

```
void ObjectPlacementSystem.onCustomPrefabSaveButtonClick (
    savingCustomObjects customObject)
```

Saves a custom object configuration.

Parameters

<i>customObject</i>	Object configuration to save
---------------------	------------------------------

SelectObject()

```
void ObjectPlacementSystem.SelectObject (
    GameObject prefab,
    savingCustomObjects obj = null)
```

Selects an object for placement and creates a preview instance.

Parameters

<i>prefab</i>	The prefab to place
<i>obj</i>	Optional saved object configuration

In delete mode, triggers deletion flow instead of placement

SetObjectComponentsEnabled()

```
void ObjectPlacementSystem.SetObjectComponentsEnabled (
    GameObject obj,
    bool enabled) [static]
```

Enables/disables all components on an object except **ObjectID** (p. ??).

Parameters

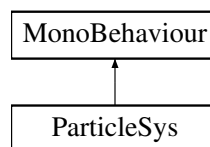
	<i>obj</i>	Target object
	<i>enabled</i>	Enable state

The documentation for this class was generated from the following file:

- C:/Repo/RPG-Table/RPG-Table/Assets/Features/PlayerCardSystem/Scripts/ObjectPlacementSystem.cs

1.30 ParticleSys Class Reference

Inheritance diagram for ParticleSys:

**Public Attributes**

- bool **Adventage** = false
- bool **Disadvantage** = false
- bool **Crawl** = false
- bool **Overwatch** = false
- bool **Block** = false
- bool **Dead** = false
- bool **Stunned** = false
- bool **Stealth** = false
- bool **Cover** = false

The documentation for this class was generated from the following file:

- C:/Repo/RPG-Table/RPG-Table/Assets/Features/Particles/Scripts/ParticleSystem.cs

1.31 ObjectPlacementSystem.PlaceableObject Class Reference

Serializable class representing a placeable object.

Public Attributes

- string **name**
- GameObject **prefab**
- string **imageName**

1.31.1 Detailed Description

Serializable class representing a placeable object.

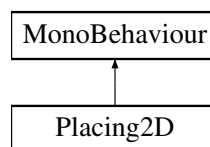
The documentation for this class was generated from the following file:

- C:/Repo/RPG-Table/RPG-Table/Assets/Features/PlayerCardSystem/Scripts/ObjectPlacementSystem.cs

1.32 Placing2D Class Reference

Handles 2D asset placement in the UI layer system.

Inheritance diagram for Placing2D:



Public Member Functions

- void **PlaceAsset** ()
Places the configured asset in the game world.

Public Attributes

- string **asset**
- GameObject **GameManager**
- GameObject **imagePrefab**
- GameObject **placedObject**

1.32.1 Detailed Description

Handles 2D asset placement in the UI layer system.

Features:

- Loads and places 2D assets from files
- Manages placement preview with mouse tracking
- Handles placement cancellation
- Integrates with **LayerSystem** (p. ??) for proper positioning

1.32.2 Member Function Documentation

PlaceAsset()

```
void Placing2D.PlaceAsset ()
```

Places the configured asset in the game world.

- Validates asset path
- Loads texture from file
- Creates UI Image with proper sprite
- Adds necessary components (**SmartDragHandler** (p. ??), **AssetName** (p. ??))
- Positions at mouse location

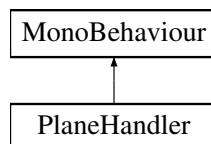
The documentation for this class was generated from the following file:

- C:/Repo/RPG-Table/RPG-Table/Assets/Features/AdditilanScripts/Placing2D.cs

1.33 PlaneHandler Class Reference

Handles terrain modification including elevation, flattening, holes, and texture changes. Provides tools for user interaction and terrain data import/export.

Inheritance diagram for PlaneHandler:



Public Member Functions

- void **ChangeBrushSize** ()
- void **GiveTerrain** (Terrain newTerrain)
Sets a new terrain reference.
- void **ExportTerrain** (string outputPath)
Exports terrain data (height and holes) to a JSON file.
- void **ImportTerrain** (string inputPath)
Imports terrain data (height and holes) from a JSON file.
- void **ClearTerrain** ()
Resets terrain elevation and holes.
- void **ChangeTerrainTexture** (string imagePath)
Starts the process to change the terrain texture from a local file.

Public Attributes

- Terrain **terrain**
Reference to the active terrain object.
- string **terrainTexturePath**
Path to texture for terrain.
- Toggle **fillTexture**
Switch for filling the terrain texture.
- Toggle **editModeOn**
Switch for edit mode.
- Slider **brushSizeSlider**
Slider variable for brush size.

1.33.1 Detailed Description

Handles terrain modification including elevation, flattening, holes, and texture changes. Provides tools for user interaction and terrain data import/export.

1.33.2 Member Function Documentation

ChangeTerrainTexture()

```
void PlaneHandler.ChangeTerrainTexture (
    string imagePath)
```

Starts the process to change the terrain texture from a local file.

Parameters

imagePath	Full path to the image file.
--	------------------------------

ExportTerrain()

```
void PlaneHandler.ExportTerrain (
    string outputPath)
```

Exports terrain data (height and holes) to a JSON file.

Parameters

outputPath	The file path to write to.
---	----------------------------

GiveTerrain()

```
void PlaneHandler.GiveTerrain (
    Terrain newTerrain)
```

Sets a new terrain reference.

Parameters

<code>newTerrain</code>	The new terrain to use.
-------------------------	-------------------------

ImportTerrain()

```
void PlaneHandler.ImportTerrain (  
    string inputPath)
```

Imports terrain data (height and holes) from a JSON file.

Parameters

<code>inputPath</code>	The file path to read from.
------------------------	-----------------------------

The documentation for this class was generated from the following file:

- C:/Repo/RPG-Table/RPG-Table/Assets/Features/BoardEditor/PlaneHandler.cs

1.34 PlayercardNotes Class Reference

Serializable container for player card notes.

Public Attributes

- string **Notes**
The text content of player notes.

1.34.1 Detailed Description

Serializable container for player card notes.

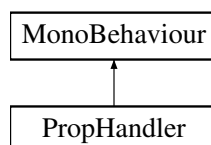
The documentation for this class was generated from the following file:

- C:/Repo/RPG-Table/RPG-Table/Assets/Features/AdditilanScripts/LoadPlayerCard.cs

1.35 PropHandler Class Reference

Handles the interaction logic for props including spawning, dragging, rotating, scaling, elevation, bloom toggle, and intensity adjustments.

Inheritance diagram for PropHandler:



Public Member Functions

- void **SetObjectToSpawn** (GameObject newObject)
Sets the object to be spawned.
- void **SetObjectToSpawnByName** (string prefabName)
Loads a prefab from Resources by name and sets it as the object to spawn.
- void **HandleSpawnProp** ()
Instantiates the selected objectToSpawn at the mouse position when R is pressed.
- void **LoadOBJFromPath** (string filePath)
Loads an OBJ model from a file path and sets it as the object to spawn.

Public Attributes

- GameObject **objectToSpawn**
The object (prefab or loaded model) to spawn.
- bool **spawnActive**
Switch for spawning props.
- string **spawnObjectName**
Holds name for object to spawn.
- GameObject **currentLayer**
Contains information about current layer on which objects are spawned.
- GameObject **particleSystemPrefab**
Contains prefab for particle system.

1.35.1 Detailed Description

Handles the interaction logic for props including spawning, dragging, rotating, scaling, elevation, bloom toggle, and intensity adjustments.

1.35.2 Member Function Documentation

LoadOBJFromPath()

```
void PropHandler.LoadOBJFromPath (
    string filePath)
```

Loads an OBJ model from a file path and sets it as the object to spawn.

Parameters

<code>filePath</code>	Full path to the .obj file.
-----------------------	-----------------------------

SetObjectToSpawn()

```
void PropHandler.SetObjectToSpawn (
    GameObject newObject)
```

Sets the object to be spawned.

Parameters

<code>newObject</code>	The new object prefab or model.
------------------------	---------------------------------

SetObjectToSpawnByName()

```
void PropHandler.SetObjectToSpawnByName (
    string prefabName)
```

Loads a prefab from Resources by name and sets it as the object to spawn.

Parameters

<code>prefabName</code>	Name of the prefab in the Resources folder.
-------------------------	---

The documentation for this class was generated from the following file:

- C:/Repo/RPG-Table/RPG-Table/Assets/Features/BoardEditor/PropHandler.cs

1.36 SaveData Class Reference

Top-level container for saved layout data.

Public Attributes

- string **saveTime**
Timestamp of save.
- int **childCount**
Number of child objects.
- List< **ChildData** > **children**
List of child data.

1.36.1 Detailed Description

Top-level container for saved layout data.

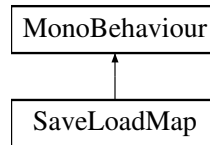
The documentation for this class was generated from the following file:

- C:/Repo/RPG-Table/RPG-Table/Assets/Features/PlayerCardSystem/Scripts/SaveLoad.cs

1.37 SaveLoadMap Class Reference

Manages map saving and loading operations for both 2D and 3D modes.

Inheritance diagram for SaveLoadMap:



Public Member Functions

- void **createMap** ()
Creates a new map with specified parameters.
- void **ClearMap** ()
Completely clears the current map state.
- void **saveMap** ()
Saves current map state to file.
- void **loadMap** (string mName)
Loads a map from file.
- void **OpenCreateMapWindow** ()
Opens the map creation UI window.
- void **CancelMapCreation** ()
Cancels map creation and resets UI.

Public Attributes

- GameObject **mapTypeWhileCreatingMap**
- GameObject **mapNameWhileCreatingMap**
- GameObject **CreateMapWindow**
- GameObject **camera**
- GameObject **GameManager**
- Transform **terrain**
- string **mapName**

1.37.1 Detailed Description

Manages map saving and loading operations for both 2D and 3D modes.

Handles:

- Map creation and serialization
- Layer-based object storage
- Automatic periodic saving
- Terrain and skybox management
- Drawing data persistence

1.37.2 Member Function Documentation

ClearMap()

```
void SaveLoadMap.ClearMap ()
```

Completely clears the current map state.

Handles both 2D and 3D cases:

- 2D: Clears all layer objects and drawings
- 3D: Clears objects and resets terrain/skybox

createMap()

```
void SaveLoadMap.createMap ()
```

Creates a new map with specified parameters.

- Creates map info file
- Clears existing map
- Loads new empty map
- Refreshes map selection UI

loadMap()

```
void SaveLoadMap.loadMap (  
    string mName)
```

Loads a map from file.

Parameters

<u>mName</u>	Map filename to load
--------------	----------------------

- Automatically saves current map before loading
- Handles both 2D and 3D map types
- Restores all layer objects
- Loads additional data (drawings/terrain)

saveMap()

```
void SaveLoadMap.saveMap ()
```

Saves current map state to file.

Saves different data based on mode:

- 2D: Layer objects + drawing data
- 3D: Layer objects + terrain data + skybox

The documentation for this class was generated from the following file:

- C:/Repo/RPG-Table/RPG-Table/Assets/Features/AdditilanScripts/SaveLoadMap.cs

1.38 SavePrefabData Class Reference

Data structure for saving prefab configurations.

Public Attributes

- string **saveTime**
- int **prefabCount**
- List< **savingCustomObjects** > **prefabs**

1.38.1 Detailed Description

Data structure for saving prefab configurations.

The documentation for this class was generated from the following file:

- C:/Repo/RPG-Table/RPG-Table/Assets/Features/PlayerCardSystem/Scripts/ObjectPlacementystem.cs

1.39 savingCustomObjects Class Reference

Complete configuration for a saved UI object.

Public Attributes

- string **name**
- string **prefab**
- string **imageName**
- string **transparency**
- string **text**
- string **fontSize**
- string **fontColor**
- bool **isBold**
- bool **isItalic**
- string **Width**
- string **Height**

1.39.1 Detailed Description

Complete configuration for a saved UI object.

The documentation for this class was generated from the following file:

- C:/Repo/RPG-Table/RPG-Table/Assets/Features/PlayerCardSystem/Scripts/ObjectPlacementSystem.cs

1.40 ScriptData Class Reference

Serialized script data.

Public Attributes

- string **type**
Type of script.
- string **data**
Serialized script data.

1.40.1 Detailed Description

Serialized script data.

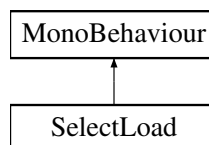
The documentation for this class was generated from the following file:

- C:/Repo/RPG-Table/RPG-Table/Assets/Features/PlayerCardSystem/Scripts/SaveLoad.cs

1.41 SelectLoad Class Reference

UI manager for loading and saving card layouts.

Inheritance diagram for SelectLoad:



Public Member Functions

- void **EnterDeleteMode** ()
Activates file deletion mode.
- void **ConfirmDelete** ()
Confirms and executes file deletion.
- void **CancelDelete** ()
Cancels pending deletion operation.
- void **onSaveButtonClick** ()
Opens the save dialog window.
- void **savingButtonClick** ()
Executes save operation with current filename.
- void **onLoadButtonClick** ()
Opens file browser and populates file list.
- void **onCancelButtonClick** ()
Closes active windows.

Public Attributes

- GameObject **objectButtonPrefab**
- GameObject **loadSelectWindow**
- GameObject **saveWindow**
- Transform **filesPanel**
- GameObject **deleteConfirmationDialog**

1.41.1 Detailed Description

UI manager for loading and saving card layouts.

Handles file selection, deletion, and save/load operations for card layouts. Manages the file browser interface and confirmation dialogs.

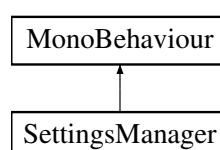
The documentation for this class was generated from the following file:

- C:/Repo/RPG-Table/RPG-Table/Assets/Features/PlayerCardSystem/Scripts/SelectSaveLoad.cs

1.42 SettingsManager Class Reference

Manages game settings including loading, saving, and STL file conversion.

Inheritance diagram for SettingsManager:



Static Public Member Functions

- static void **SaveSettings** ()
Saves current settings to a JSON file.
- static void **LoadSettings** ()
Loads settings from JSON file or creates default settings if file doesn't exist.

Static Public Attributes

- static **GameSettings _CurrentSettings**
Current game settings.

1.42.1 Detailed Description

Manages game settings including loading, saving, and STL file conversion.

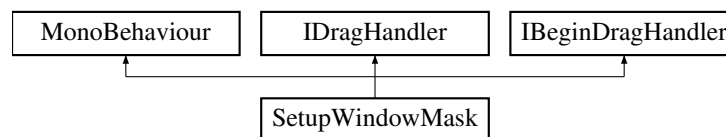
The documentation for this class was generated from the following file:

- C:/Repo/RPG-Table/RPG-Table/Assets/Features/SettingsManager.cs

1.43 SetupWindowMask Class Reference

Handles draggable masked window area for UI cards.

Inheritance diagram for SetupWindowMask:



Public Member Functions

- void **OnBeginDrag** (PointerEventData eventData)
Handles drag start event.
- void **OnDrag** (PointerEventData eventData)
Handles continuous drag movement.

1.43.1 Detailed Description

Handles draggable masked window area for UI cards.

Implements drag functionality for a masked content area. Automatically adds RectMask2D if missing and manages parent-child relationship.

1.43.2 Member Function Documentation

OnBeginDrag()

```
void SetupWindowMask.OnBeginDrag (
    PointerEventData eventData)
```

Handles drag start event.

Parameters

<code>eventData</code>	Pointer event data
------------------------	--------------------

OnDrag()

```
void SetupWindowMask.OnDrag (
    PointerEventData eventData)
```

Handles continuous drag movement.

Parameters

<code>eventData</code>	Pointer event data
------------------------	--------------------

Moves content area relative to drag delta with speed adjustment

The documentation for this class was generated from the following file:

- C:/Repo/RPG-Table/RPG-Table/Assets/Features/PlayerCardSystem/Scripts/SetupWindowMask.cs

1.44 SFXElement Class Reference

Represents a sound effect element that pairs an SFXType with its corresponding AudioClip.

Properties

- **SFXType Type** [get]
Gets the type identifier for this sound effect.
- **AudioClip Clip** [get]
Gets the AudioClip associated with this sound effect.

1.44.1 Detailed Description

Represents a sound effect element that pairs an SFXType with its corresponding AudioClip.

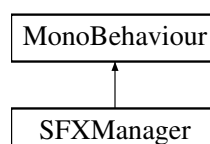
The documentation for this class was generated from the following file:

- C:/Repo/RPG-Table/RPG-Table/Assets/Features/Audio/Scripts/SFXElement.cs

1.45 SFXManager Class Reference

Manages and plays sound effects, including one-shot and looped audio.

Inheritance diagram for SFXManager:



Public Member Functions

- void **Play** (SFXType type)
Plays a one-shot sound effect of the specified type.
- void **PlayLooped** ()
Starts playing the configured looped sound effect. The sound will continue playing until explicitly stopped.
- void **StopLooped** ()
Stops the currently playing looped sound effect.

1.45.1 Detailed Description

Manages and plays sound effects, including one-shot and looped audio.

1.45.2 Member Function Documentation

Play()

```
void SFXManager.Play (
    SFXType type)
```

Plays a one-shot sound effect of the specified type.

Parameters

<code>type</code>	The type of sound effect to play, as defined in the SFXType enum.
-------------------	---

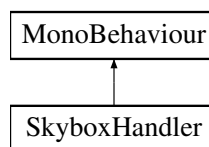
The documentation for this class was generated from the following file:

- C:/Repo/RPG-Table/RPG-Table/Assets/Features/Audio/Scripts/SFXManager.cs

1.46 SkyboxHandler Class Reference

Handles dynamic loading and application of a skybox texture at runtime.

Inheritance diagram for SkyboxHandler:



Public Member Functions

- void **ChangeSkybox** (string texturePath)
Changes the current skybox using the texture at the specified path.
- void **ClearSkybox** ()

Public Attributes

- Material **skyboxMaterialTemplate**
Template material used for setting the loaded skybox texture.
- string **skyboxTexturePath**

1.46.1 Detailed Description

Handles dynamic loading and application of a skybox texture at runtime.

1.46.2 Member Function Documentation

ChangeSkybox()

```
void SkyboxHandler.ChangeSkybox (
    string texturePath)
```

Changes the current skybox using the texture at the specified path.

Parameters

<code>texturePath</code>	Absolute path to the texture file.
--------------------------	------------------------------------

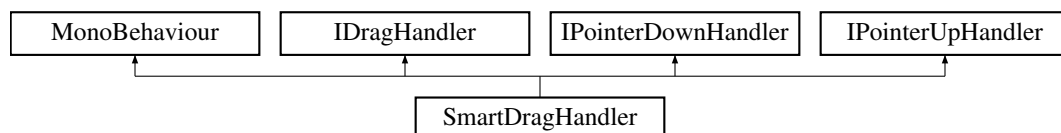
The documentation for this class was generated from the following file:

- C:/Repo/RPG-Table/RPG-Table/Assets/Features/BoardEditor/SkyboxHandler.cs

1.47 SmartDragHandler Class Reference

Handles advanced drag functionality for UI elements, including multi-select and global drag operations. Implements IDragHandler, IPointerDownHandler, and IPointerUpHandler to respond to Unity's drag events.

Inheritance diagram for SmartDragHandler:



Public Member Functions

- void **OnPointerDown** (PointerEventData eventData)
Handles pointer down events to initiate drag operations.
- void **OnDrag** (PointerEventData eventData)
Handles drag events to move the object and any selected objects.
- void **OnPointerUp** (PointerEventData eventData)
Handles pointer up events to complete drag operations.

Public Attributes

- bool **Edit** = false
Determines if the object is in edit mode (editor-only functionality).
- bool **Game** = true
Determines if the object is in game mode (runtime functionality).

Properties

- static bool **isDragging** [get]
*Indicates whether any **SmartDragHandler** (p. ??) is currently being dragged.*

1.47.1 Detailed Description

Handles advanced drag functionality for UI elements, including multi-select and global drag operations. Implements `IDragHandler`, `IPointerDownHandler`, and `IPointerUpHandler` to respond to Unity's drag events.

1.47.2 Member Function Documentation

OnDrag()

```
void SmartDragHandler.OnDrag (  
    PointerEventData eventData)
```

Handles drag events to move the object and any selected objects.

Parameters

	<i>eventData</i>	Pointer event data containing current position information.
---	------------------	---

OnPointerDown()

```
void SmartDragHandler.OnPointerDown (  
    PointerEventData eventData)
```

Handles pointer down events to initiate drag operations.

Parameters

	<i>eventData</i>	Pointer event data containing position and button information.
---	------------------	--

OnPointerUp()

```
void SmartDragHandler.OnPointerUp (  
    PointerEventData eventData)
```

Handles pointer up events to complete drag operations.

Parameters

<code>eventData</code>	Pointer event data containing button information.
------------------------	---

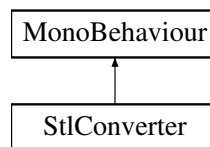
The documentation for this class was generated from the following file:

- C:/Repo/RPG-Table/RPG-Table/Assets/Features/PlayerCardSystem/Scripts/SmartDragHandler.cs

1.48 StlConverter Class Reference

Converts STL files (ASCII or binary) to Unity Meshes and exports them as OBJ files.

Inheritance diagram for StlConverter:



Public Member Functions

- void **Convert** (string filePath, string exportPath)
Converts an STL file into a Unity Mesh and exports it as an OBJ file.

1.48.1 Detailed Description

Converts STL files (ASCII or binary) to Unity Meshes and exports them as OBJ files.

1.48.2 Member Function Documentation

Convert()

```
void StlConverter.Convert (
    string filePath,
    string exportPath)
```

Converts an STL file into a Unity Mesh and exports it as an OBJ file.

Parameters

<code>filePath</code>	Path to the input STL file.
<code>exportPath</code>	Path to save the output OBJ file.

The documentation for this class was generated from the following file:

- C:/Repo/RPG-Table/RPG-Table/Assets/Features/BoardEditor/StlConverter.cs

1.49 TerrainData Class Reference

Serializable class that holds terrain-related data including height and hole maps.

Public Attributes

- float **brushDefaultSave**
The default brush strength value used for terrain editing or painting.
- int **heightmapWidth**
Width of the heightmap in pixels or units.
- int **heightmapHeight**
Height of the heightmap in pixels or units.
- int **holemapWidth**
Width of the holemap, typically matching the terrain tile's resolution.
- int **holemapHeight**
Height of the holemap, typically matching the terrain tile's resolution.
- List< float > **heightMap**
A flat list representing the height values of the terrain. Typically normalized between 0 and 1.
- List< bool > **holeMap**
A flat list representing holes in the terrain. True indicates a hole; False indicates solid terrain.

1.49.1 Detailed Description

Serializable class that holds terrain-related data including height and hole maps.

The documentation for this class was generated from the following file:

- C:/Repo/RPG-Table/RPG-Table/Assets/Features/BoardEditor/TerrainData.cs